

Name: Krish Singh

Roll: AID25072

B.Tech Artificial Intelligence & Data Science - 2nd Semester

Object Oriented Programming – Lab Assignment 9

General Questions:

1. What problem is observed in normal class design?

Ans: The problem observed in normal class design is that it provides a single generic implementation for all subclasses, which leads to incorrect behavior when different subclasses require different logic based on real-world scenarios.

2. How does an abstract class improve correctness?

Ans: Abstract class improves correctness by enforcing subclasses to implement specific methods while still allowing shared code, ensuring that each subclass provides its own appropriate behavior for that method maintaining behavior correctness and realistic outcomes.

3. How does an interface differ from an abstract class?

Ans: An interface only defines what a class should do by declaring methods without any implementation, whereas an abstract class can both define methods and provide some common implementation that subclasses can directly use.

4. Which approach enforces implementation more strictly?

Ans: An interface enforces implementation more strictly because all its methods must be implemented by the class that implements it, whereas an abstract class can have both implemented and unimplemented methods.

5. Which design is more flexible for future extension?

Ans: The interface-based design is more flexible for future extension because a class can implement multiple interfaces, allowing greater adaptability and modularity in system design.

Conceptual Questions:

1. When would you prefer an abstract class over an interface?

Ans: I would prefer an abstract class when I am working with related classes that need to share some common code or data, and at the same time I want to ensure that certain methods are implemented differently by each subclass.

2. When is an interface more suitable?

Ans: An interface is more suitable when you want to define a set of methods that different classes must implement, without providing any shared code, especially when those classes are not closely related and may need to follow the same set of rules in different ways.

3. Can a class implement multiple interfaces? Explain.

Ans: Yes, a class can implement multiple interfaces, and this allows the class to inherit methods from different sources, thereby supporting a type of multiple inheritance and increasing flexibility in design.

Exercise 1:

Normal Class

Code:

```
class Payment{
    double amount;
    void pay(){
        System.out.println("Processing Payment of "+amount);
    }
}

public class PaymentProcessingPartA {
    public static void main(String[] args) {
        Payment p = new Payment();
        p.pay();
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/
Processing Payment of 0.0

Process finished with exit code 0
```

Abstract Class

Code:

```
abstract class PaymentB{
    double amount;
    PaymentB(double amt){
        this.amount=amt;
    }
    abstract void pay();
}

class UPI extends PaymentB{
    String upiId;
    UPI(double amt,String upiId){
        super(amt);
        this.upiId=upiId;
    }
    void pay(){
        System.out.println("UPI payment : "+amount+" using "+upiId);
    }
}
```

```

public class PaymentProcessingPartB {
    public static void main(String[] args) {
        PaymentB p = new UPI(5000,"abstract@oksbi");
        p.pay();
    }
}

```

Output:

```

/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/
UPI payment : 5000.0 using abstract@oksbi

Process finished with exit code 0

```

Interface Class

Code:

```

interface PaymentC{
    void pay();
}
class UPIC implements PaymentC{
    double amount;
    String upiID;
    UPIC(double amount,String upiID){
        this.amount=amount;
        this.upiID=upiID;
    }
    public void pay(){
        System.out.println("UPI Payment : "+amount+" using "+upiID);
    }
}
public class PaymentProcessingPartC{
    public static void main(String[] args){
        PaymentC p = new UPIC(500,"interface@oksbi");
        p.pay();
    }
}

```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share,  
UPI Payment : 500.0 using interface@oksbi
```

```
Process finished with exit code 0
```

Exercise 2:

Abstract Class

Code:

```
abstract class Employee{
    double base;
    Employee(double base){
        this.base=base;
    }
    abstract void calculateSalary();
}
class FullTime extends Employee{
    FullTime(double base){
        super(base);
    }
    void calculateSalary(){
        System.out.println("Salary : "+(base+5000));
    }
}
public class EmployeeSalaryAbstract{
    public static void main(String[] args){
        Employee e = new FullTime(35000);
        e.calculateSalary();
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/
Salary : 40000.0

Process finished with exit code 0
```

Interface Class

Code:

```
interface EmployeeB{
    void calculateSalary();
}
class Intern implements EmployeeB{
    double stipend;
    Intern(double stipend){
        this.stipend=stipend;
    }
}
```

```
    public void calculateSalary(){
        System.out.println("Stipend : "+stipend);
    }
}
public class EmployeeSalaryInterface {
    public static void main(String[] args) {
        EmployeeB e = new Intern(55050.5050);
        e.calculateSalary();
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/
Stipend : 55050.505

Process finished with exit code 0
```

Exercise 3:

Abstract Class

Code:

```
abstract class Notification{
    abstract void send();
}
class Email extends Notification{
    void send(){
        System.out.println("Sending email...");
    }
}
public class NotificationSystemAbstract {
    public static void main(String[] args) {
        Notification n = new Email();
        n.send();
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/
Sending email...

Process finished with exit code 0
```

Interface Class

Code:

```
interface Notificationn{
    void send();
}
class SMS implements Notificationn{
    public void send(){
        System.out.println("Sending SMS");
    }
}
public class NotificationSystemInterface {
    public static void main(String[] args) {
        Notificationn n = new SMS();
        n.send();
    }
}
```


Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/  
Sending SMS  
  
Process finished with exit code 0
```

Exercise 4:

Abstract Class

Code:

```
abstract class Vehicle{
    double distance;
    Vehicle(double distance){
        this.distance = distance;
    }
    abstract void fare();
}
class Car extends Vehicle{
    Car(double distance){
        super(distance);
    }
    void fare(){
        System.out.println("Car Fare: "+distance*15);
    }
}
public class VehicleFareAbstract {
    public static void main(String[] args) {
        Vehicle v = new Car(100);
        v.fare();
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/
Car Fare: 1500.0

Process finished with exit code 0
```

Interface Class

Code:

```
interface Vehiclee{
    void fare();
}
class Bike implements Vehiclee{
    double distance;
    Bike(double distance){
        this.distance = distance;
    }
}
```

```
}  
public void fare(){  
    System.out.println("Bike Fare : "+distance*8);  
}  
}  
public class VehicleFareInterface {  
    public static void main(String[] args) {  
        Vehicle v = new Bike(100);  
        v.fare();  
    }  
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share/  
Bike Fare : 800.0  
  
Process finished with exit code 0
```